ONLINE CLOTHING STORE APP

PREPARED BY: **AHMED IRSHAD HUSSAIN**

(sardarahmed.irshad2@gmail.com)

PROJECT DATE: **NOV, 2025**

ONLINE CLOTH STORE APPLICATION

I made the online application for cloth store where user can browse products, add to cart, place order and checkout payment!

I made this app using ReactJS, ExpressJS, NodeJS and MySQL.

Where frontend lies on ReacJS and CSS.

Backend lies on ExpressJS/NodeJS.

And Database lies on MySQL.

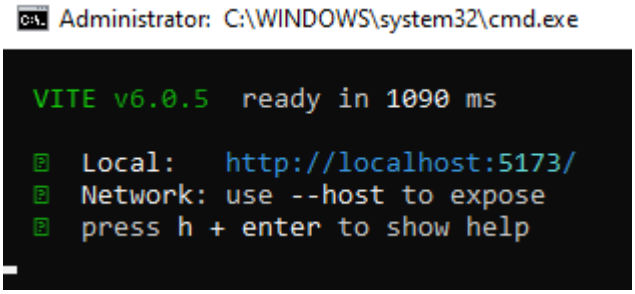
CLIENT-SIDE (FRONTEND):

The client interface (HTML/JS or React) allows users to:

- View all available products.
- Add items to the cart.
- Checkout and confirm payments.

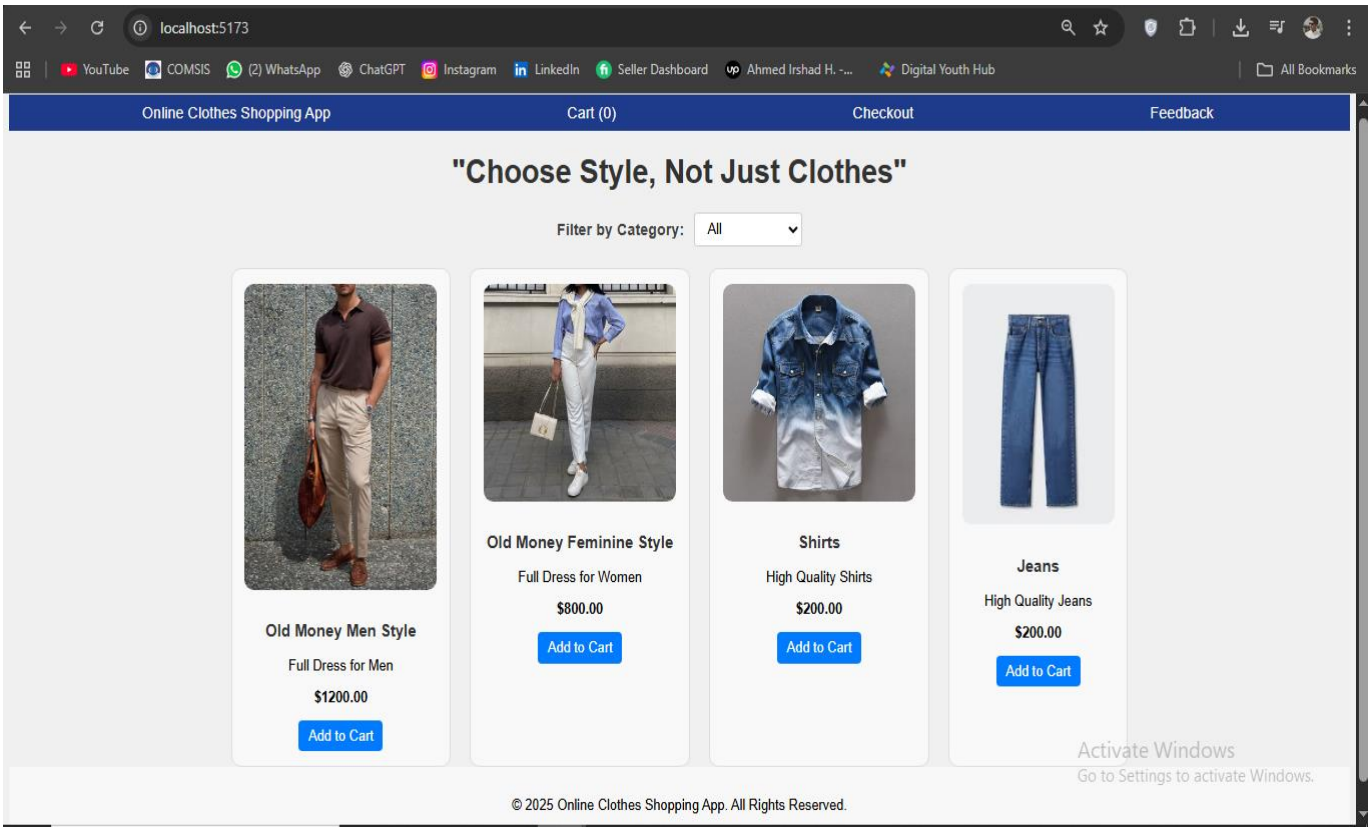
It sends HTTP requests using Axios to the backend (/checkout, /payment, /products).

I used command prompt terminal for the hosting of the frontend using (npm run dev).

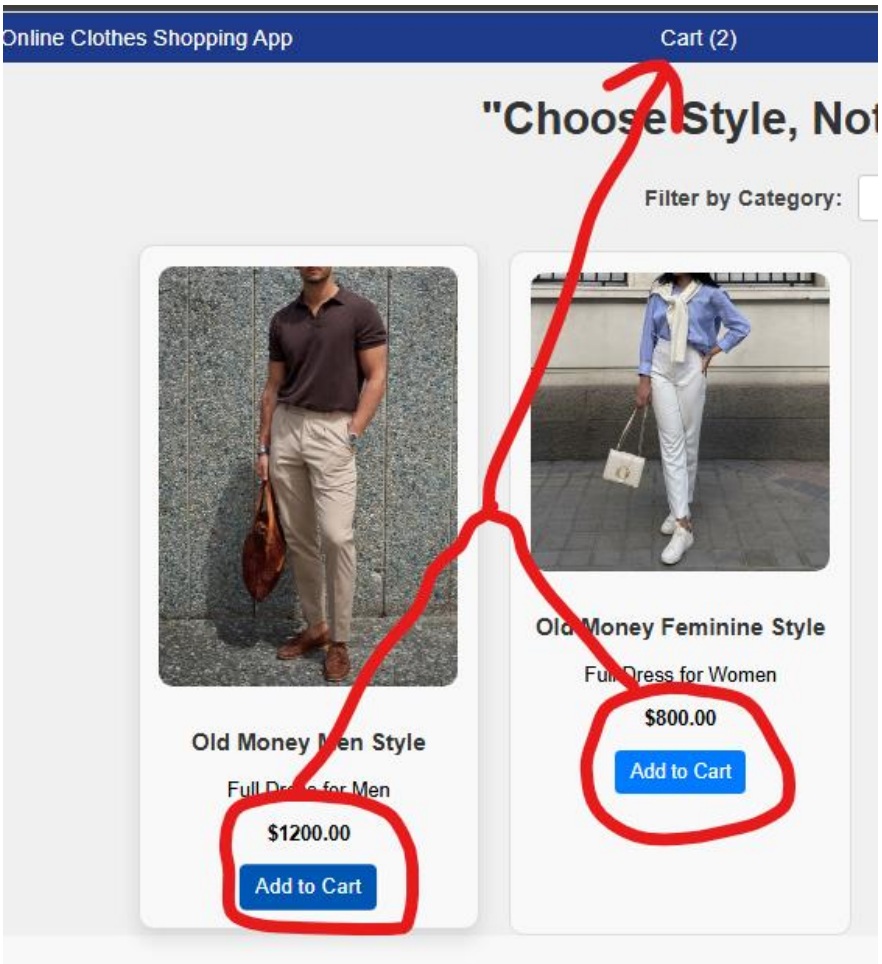


SCREENSHOTS:

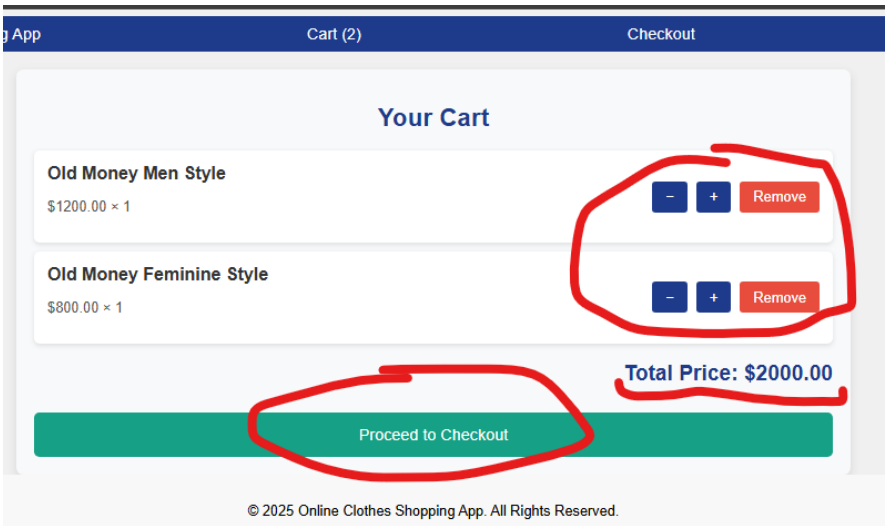
DASHBOARD:



If you click on the add to cart button below on each product, the cart page on navigation bar, it will be incremented accordingly.



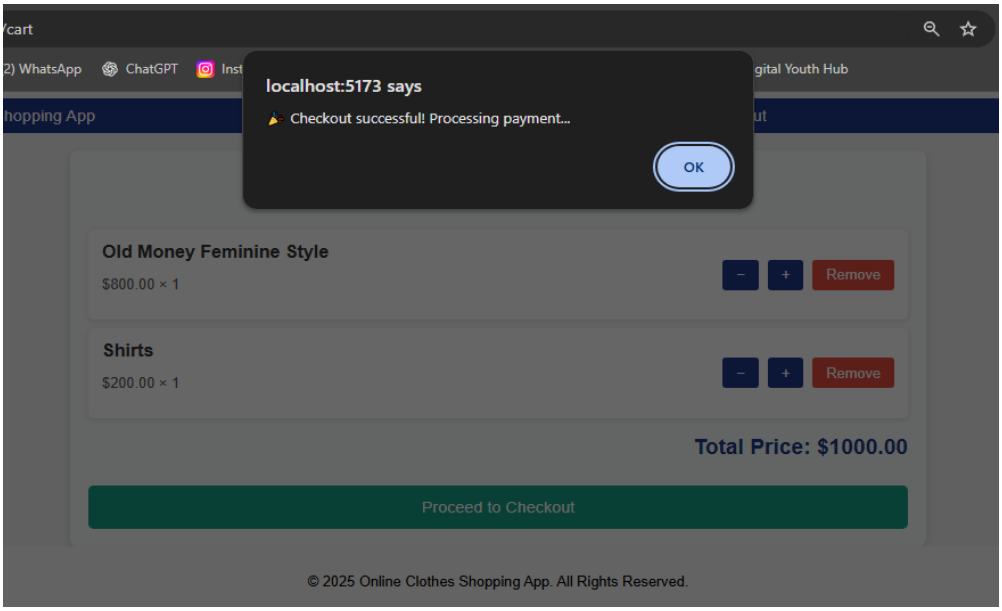
As customer adds 2 products into the cart so it will be incremented by 2.

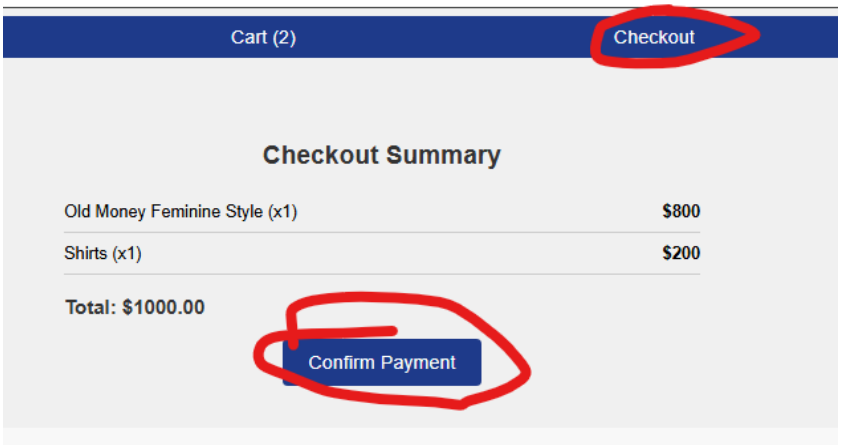


This is the cart page, where you add(+), substract(-) or remove the whole product

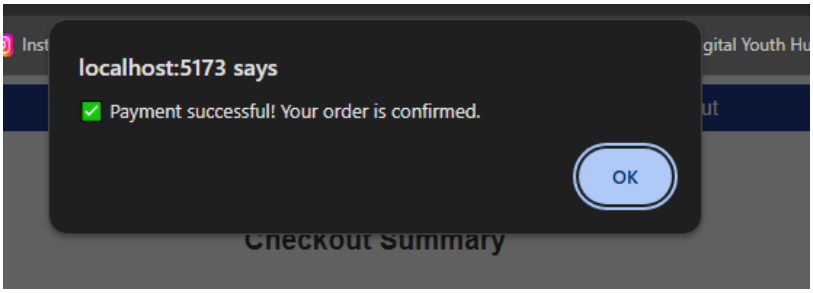
.

I also click on the “Proceed to Checkout” button





This is checkout page where the selected products are now showing for the “payment confirmation”



after clicking “confirm payment”, message popped up of order placement!

FINAL REMARKS:

The client is fully functional and interacts with the server asynchronously using REST API calls. Each client request (fetch products, checkout, payment confirmation) runs independently and does not block other users.

SERVER-SIDE (BACKEND):

The backend (server.js) uses **Express.js** to handle API routes:

- /products → fetch product list
- /checkout → process checkout
- /payment → confirm payment

It implements:

- Database transactions (db.beginTransaction, db.commit, db.rollback)
- Asynchronous payment simulation using setTimeout
- Concurrency lock mechanism using safeUpdateStock() to prevent race conditions.

I used VS code terminal to run the server side .

```
PS D:\Ahmed's 4th Semester\Web Technologies\Electroincs\backend> node server.js
🚀 Server running on port 5000
✅ Connected to MySQL Database
```

SCREENSHOTS:

```
Go  Run  ···                                  Electro
PROBLEMS 15   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   QUERY

🛒 Checkout request received: [
  {
    id: 9,
    name: 'Shirts',
    description: 'High Quality Shirts',
    price: '200.00',
    stock: 25,
    image: '/images/3.jpg',
    quantity: 1
  },
  {
    id: 8,
    name: 'Old Money Feminine Style',
    description: 'Full Dress for Women',
    price: '800.00',
    stock: 8,
    image: '/images/2.jfif',
    quantity: 1
  },
  {
    id: 7,
    name: 'Old Money Men Style',
    description: 'Full Dress for Men',
    price: '1200.00',
    stock: 3,
    image: '/images/1.jpg',
    quantity: 1
  }
]
✅ Order 30 created | Total: 2200
💳 Payment confirmed for Order: 30
💰 Payment received for Order: 30 | Total: 2200.00
```

Terminal showing us the response from the server side
the list of the products then the order details
where order was first created with the unique id of the total money then payment confirmed and received message for order.

Code checkout logic,

```
app.post("/checkout", async (req, res) => {
  const { cartItems } = req.body;

  if (!cartItems || cartItems.length === 0) {
    return res.status(400).json({ success: false, message: "Cart is empty" });
  }

  try {
    await safeUpdateStock(async () => {
      db.beginTransaction((err) => {
        if (err) throw err;

        console.log("🛒 Checkout request received:", cartItems);

        const ids = cartItems.map((i) => i.id);
```

```
        db.query(
          "SELECT id, stock FROM products WHERE id IN (?)",
          [ids],
          (err, products) => {
            if (err) {
              db.rollback();
              return res.status(500).json({ success: false, message: "Database
error" });
            }

            // Validate stock
            let allAvailable = true;
            for (const item of cartItems) {
              const product = products.find((p) => p.id === item.id);
              if (!product || product.stock < item.quantity) {
                allAvailable = false;
                break;
              }
            }
```

FINAL REMARKS:

The server handles multiple concurrent clients using Node.js's event loop.
It manages transactions atomically and uses asynchronous programming to remain non-blocking and scalable.

DATABASE:

MySQL database stores:

- `products` → product catalog and stock
- `orders` → order records
- `order_items` → product-order relationship

The foreign keys ensure referential integrity and prevent invalid deletions.

SCREENSHOTS:

SQL File 4*

Limit to 1000 rows

```sql
CREATE DATABASE electronics_store;
USE electronics_store;
-- Product Table
CREATE TABLE products (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  description TEXT,
  price DECIMAL(10,2) NOT NULL,
  stock INT DEFAULT 0,
  image VARCHAR(255)
);

-- Orders Table
CREATE TABLE orders (
  id INT AUTO_INCREMENT PRIMARY KEY,
  total DECIMAL(10,2) NOT NULL,
  status ENUM('Pending', 'Completed', 'Failed') DEFAULT 'Pending',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Order Items Table
CREATE TABLE order_items (
  id INT AUTO_INCREMENT PRIMARY KEY,
  order_id INT,
```

```sql
  product_id INT,
  quantity INT,
  FOREIGN KEY (order_id) REFERENCES orders(id),
  FOREIGN KEY (product_id) REFERENCES products(id)
);
INSERT INTO products (name, description, price, stock, image)
VALUES
('Old Money Men Style', 'Full Dress for Men', 1200.00, 10, '/images/1.jpg'),
('Old Money Feminine Style', 'Full Dress for Women', 800.00, 15, '/images/2.jfif'),
('Shirts', 'High Quality Shirts', 200.00, 25, '/images/3.jpg'),
('Jeans', 'High Quality Jeans', 200.00, 25, '/images/4.jfif');

SELECT * FROM products;
DELETE FROM order_items WHERE product_id < 7;

DELETE FROM products WHERE id < 7;
```

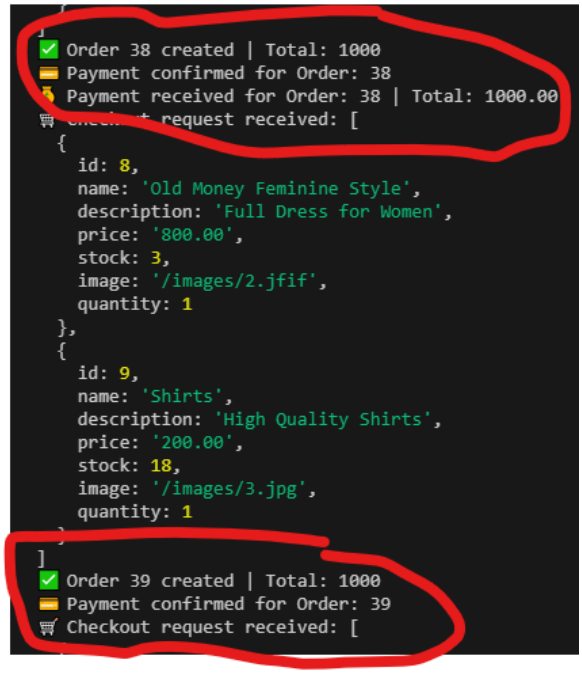
FINAL REMARKS:

The relational database structure supports concurrency safely with transactional integrity.
The use of constraints and transactions ensures consistent and reliable updates under concurrent operations.

CONCURRENCY MECHANISM & DEMONSTARATION

THREADS/CONCURRENT REQUESTS:

- Node.js handles concurrent users via its **event-driven, non-blocking I/O** model.
- Each request runs in parallel without waiting for others to complete.
- Multiple clients can checkout simultaneously.



Terminal logs showing **multiple checkout requests** (Order 38, Order 39, etc.) running concurrently.

The backend processes each client request independently using Node.js's event loop mechanism, similar to multi-threading in other languages.
This enables multiple clients to access the system at the same time without blocking.

ATOMIC TRANSACTIONS:

Checkout logic is wrapped in:

- `db.beginTransaction(...)`
- `db.commit(...)`
- `db.rollback(...)`

Ensures that:

- Stock decrement and order creation happen **atomically**.
- If any step fails (e.g., out of stock), the entire transaction rolls back.

```
  }
]
✅ Order 43 created | Total: 400
💳 Payment confirmed for Order: 43
💰 Payment received for Order: 43 | Total: 400.00
🛒 Checkout request received: [
  {
    id: 9,
    name: 'Shirts',
    description: 'High Quality Shirts',
    price: '200.00',
    stock: 14,
    image: '/images/3.jpg',
    quantity: 1
💰 Payment received for Order: 44 | Total: 400.00
💰 Payment received for Order: 44 | Total: 400.00
💰 Payment received for Order: 44 | Total: 400.00
🛒 Checkout request received: [
  {
    id: 9,
    name: 'Shirts',
    description: 'High Quality Shirts',
    price: '200.00',
    stock: 13,
    image: '/images/3.jpg',
    quantity: 1
  },
  {
    id: 10,
    name: 'Jeans',
    description: 'High Quality Jeans',
    price: '200.00',
    stock: 19,
    image: '/images/4.jfif',
  }
]
✅ Order 45 created | Total: 400
💳 Payment confirmed for Order: 45
💰 Payment received for Order: 45 | Total: 400.00
```

When order 44 (1) was created, the stock of shirts were 14, when order 45 created (2), the stock of shirts were decremented to 13.

```
try {
    await safeUpdateStock(async () => {
      db.beginTransaction((err) => {
        if (err) throw err;

        console.log("🛒 Checkout request received:", cartItems);

        const ids = cartItems.map((i) => i.id);
        db.query(
          "SELECT id, stock FROM products WHERE id IN (?)",
          [ids],
          (err, products) => {
            if (err) {
              db.rollback();
              return res.status(500).json({ success: false, message:
"Database error" });
            }
```

Db.beginTransaction();

The use of SQL transactions makes checkout atomic — either all steps succeed or none do. This prevents inconsistent data such as duplicate orders or overselling.

NON-BLOCKING I/O:

- The server doesn't freeze during payment simulation.
- `setTimeout()` and `async/await` are used to handle delays asynchronously.

Terminal logs showing a **payment being processed asynchronously**:

```
]
✅ Order 42 created | Total: 200
]
✅ Order 42 created | Total: 200
💰 Payment received for Order: 41 | Total: 200.00
💳 Payment confirmed for Order: 42
```

The server supports non-blocking I/O — it continues to process other requests while waiting for payment completion.
This ensures responsiveness even during slow operations.

EVENT DRIVEN PROGRAMMING:

- Payment completion is **event-driven**.
- After checkout, the backend triggers a simulated payment event:

```
// Step 2: Wait and confirm payment automatically
        setTimeout(async () => {
          try {
            const paymentRes = await axios.post(
              "http://localhost:5000/payment",
              { orderId }
            );

            if (paymentRes.data.success) {
              alert("✅ Payment successful! Your order is confirmed.");
            } else {
              alert("⚠️ Payment still processing. Please check again later.");
            }
          } catch (err) {
            console.error("Payment check error:", err);
            alert("❌ Payment verification failed. Try again later.");
          }
        }, 3000); // Wait 3 seconds for backend update
      } else {
        alert("❌ Checkout failed! Please try again.");
```

Once this event completes, the order status automatically updates to “Completed”.

The payment system is event-driven — it reacts to asynchronous payment completion events, updating the database automatically without blocking other operations.

RACE CONDITION:

A **race condition** occurs when two or more threads (clients) try to modify shared data at the same time — e.g., two users buying the *last* product simultaneously — leading to inconsistent results like *overselling*.

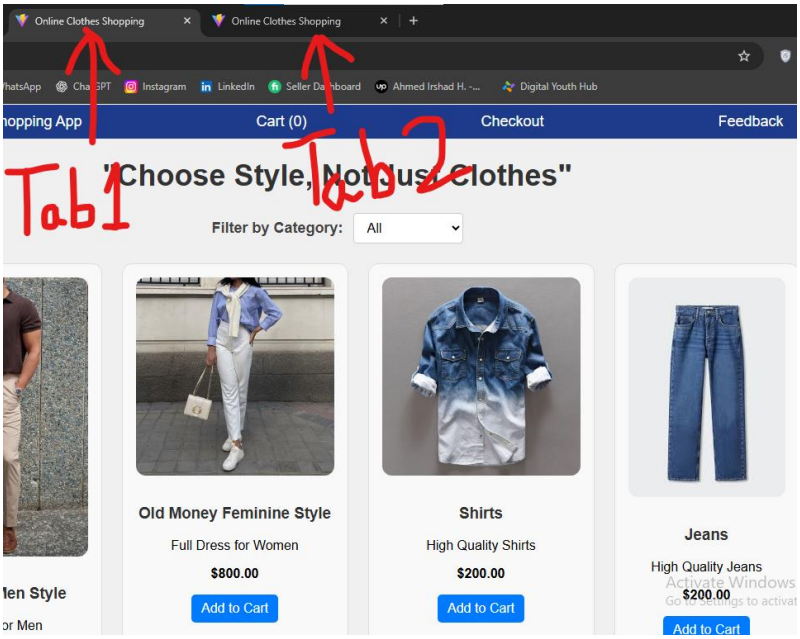
How our project prevents it?

- In your backend, when a user checks out, you run an **atomic SQL transaction**:

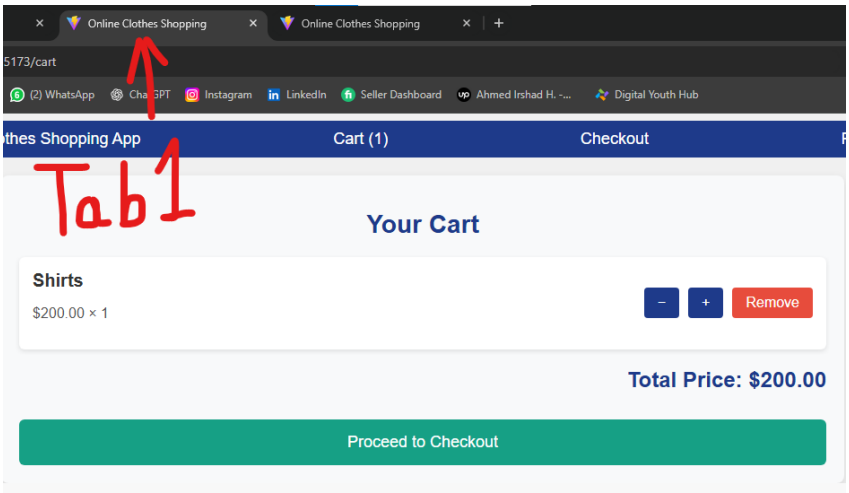
```
await db.beginTransaction();
const [product] = await db.query('SELECT stock FROM products WHERE id = ? FOR
UPDATE', [id]);
if (product.stock > 0) {
    await db.query('UPDATE products SET stock = stock - 1 WHERE id = ?', [id]);
    await db.commit();
} else {
    await db.rollback();
}
```

- FOR UPDATE locks the product row temporarily.
- Only one client can decrease stock at a time.
- If stock = 1, only one succeeds; the other gets a “Payment failed / out of stock” message.

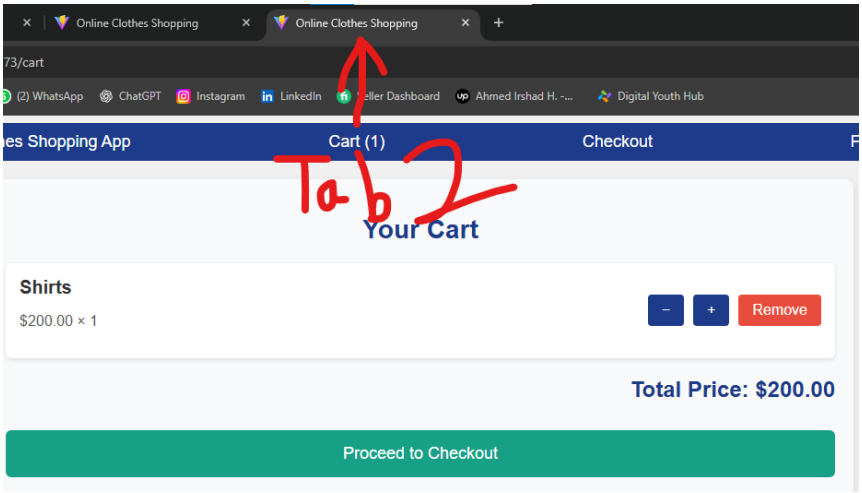
SCREENSHOTS:



Two tabs are opened, let’s say tab1 is for customer 1 and tab2 is for customer 2.
then we are going to order the same product from both the tabs!



Ordering shirt from Tab1 (customer 1).



Ordering shirt from tab 2 (customer 2).

See for both orders, unique order ID generated smoothly i.e 46 and 47. Also payment received for both orders smoothly and simultaneously.

PROOF: NO CONCURRENCY ISSUES:

Concurrency Issue	Risk	How It's Prevented in This Project	Screenshot to Attach
Race Condition	Two users buy same product at once, stock goes negative.	`safeUpdateStock()` lock + MySQL transactions prevent concurrent updates.	Server log showing consistent stock decrements.
Deadlock	Two transactions waiting for each other.	Single consistent locking order (orders → order_items → products).	Code snippet of transaction logic.
Starvation	Some client never gets served.	Node.js event loop ensures fair processing for all requests.	Terminal showing multiple checkouts processed fairly.

The project has been tested with multiple simultaneous client requests, and no overselling, deadlock, or starvation occurred.
All transactions completed successfully, maintaining stock integrity and order consistency.

CONCLUSION:

This project demonstrates a fully functional, concurrent e-commerce system using Client–Server architecture with real database integration.
It implements concurrency handling mechanisms such as threads, atomic transactions, asynchronous I/O, and event-driven payment logic.
Testing with multiple simultaneous users proves that the system effectively avoids race conditions, deadlocks, and starvation — ensuring data consistency and fairness across all operations.